

RESUME MATERI PERKULIAHAN

Perancangan dan Analisis Algoritma (A)

Pertemuan ke-3

Disusun Oleh:

Gilbran Mahdavia Raja

NRP: 5025241134

Institut Teknologi Sepuluh Nopember

Daftar Isi

1	Resume Materi Perkuliahan	2
1.1	Pelajaran dari Pertemuan Sebelumnya	2
1.2	Prinsip Worst Case Scenario	2
2	Pembahasan Kasus: SPOJ STUDCHAN2 — Student Chains 2	2
2.1	Deskripsi Permasalahan	2
2.2	Langkah 1: Menemukan Pola melalui Observasi	3
2.3	Langkah 2: Derivasi Model Matematika — Rumus $\max(k)$	3
2.4	Langkah 3: Analisis Batas dan Strategi Precompute	4
2.5	Langkah 4: Implementasi — Solusi Optimal C++	4
2.6	Kaitan STUDCHAN2 dengan CBANK: Inversi Masalah	5
3	Pembahasan Kasus: SPOJ MONONUM — Monotone Numbers	5
3.1	Deskripsi Permasalahan	5
3.2	Langkah 1: Memahami Inti Rumus dari Kode	5
3.3	Langkah 2: Dekomposisi Matematis Rumus	6
3.4	Langkah 3: Penyederhanaan via Koefisien Binomial	6
3.5	Langkah 4: Interpretasi Kombinatorik	7
3.6	Langkah 5: Verifikasi Numerik	7
3.7	Langkah 6: Kenapa Bukan Dynamic Programming?	8
3.8	Langkah 7: Analisis Kompleksitas MONONUM	8
4	Kesimpulan	8
	Bukti Submission	11

Resume Materi Perkuliahan

1.1 Pelajaran dari Pertemuan Sebelumnya

Pada pertemuan ke-2, kita telah mempelajari bahwa **observasi pola** dan **abstraksi matematis** adalah keterampilan inti dalam merancang solusi yang efisien. Studi kasus SPOJ CBANK mengajarkan alur berpikir:

Alur Berpikir Umum Soal Kombinatorika

Pahami Soal $\rightarrow$ Model Matematis $\rightarrow$ Observasi Pola $\rightarrow$ Rumus Tertutup $\rightarrow$ Kode Efisien

Pertemuan ke-3 ini memperkenalkan dua studi kasus baru:

1. **SPOJ STUDCHAN2 — Student Chains 2**: Soal ini adalah kebalikan dari CBANK. Jika CBANK diberikan N dan diminta mencari $f(N)$, maka STUDCHAN2 diberikan nilai n dan diminta mencari k terkecil yang memenuhi suatu rumus batas.
2. **SPOJ MONONUM — Monotone Numbers**: Soal dengan tag *Dynamic Programming*, namun melalui observasi mendalam dapat diselesaikan dengan rumus tertutup berbasis kombinatorik yang jauh lebih elegan dan efisien.

Pelajaran Utama Pertemuan 3

Jangan terlalu percaya pada tag kategori soal. Tag “Dynamic Programming” tidak selalu berarti DP adalah solusi terbaik. Observasi yang cermat sering kali menghasilkan solusi $O(1)$ per query yang jauh lebih cepat.

1.2 Prinsip Worst Case Scenario

Seorang pemrogram yang andal senantiasa merancang solusinya berdasarkan kondisi masukan yang paling berat. Prinsip ini selaras dengan analisis formal algoritma yang menjadikan *worst case* sebagai tolok ukur utama. Dengan mempersiapkan solusi untuk skenario terburuk, kita memperoleh jaminan bahwa program akan berfungsi dengan benar dan efisien di semua kondisi, bukan hanya pada masukan yang menguntungkan.

Pembahasan Kasus: SPOJ STUDCHAN2 — Student Chains 2

2.1 Deskripsi Permasalahan

Sejumlah siswa berdiri dalam satu garis berdekatan dan berpegangan tangan membentuk sebuah *chain* (rantai). Rantai bersifat tidak melingkar (*linear*). Diberikan bilangan bulat n (jumlah siswa), kita diminta menemukan nilai k terkecil sedemikian sehingga rantai dengan parameter k dapat menampung n siswa.

Batasan Masukan:

- $1 \leq T \leq 50$ (jumlah test case)
- $1 \leq n \leq 10^9$

Contoh masukan dan keluaran:

Input n	Output k
1	1
6	1
16	2
22	2
33	3
63	3
41	3
59	3

2.2 Langkah 1: Menemukan Pola melalui Observasi

Langkah pertama adalah membangun tabel hubungan antara nilai k dan batas maksimum siswa $\max(k)$ yang dapat ditampung:

k	$\max(k)$	Selisih terhadap $k - 1$
1	7	—
2	23	+16
3	63	+40
4	159	+96
5	383	+224

Dari tabel ini kita dapat mengamati bahwa selisihnya adalah 16, 40, 96, 224. Perhatikan rasio antar selisih berurutan: $40/16 = 2.5$, $96/40 = 2.4$, $224/96 \approx 2.33$. Selisih ini tidak membentuk rasio konstan, sehingga pendekatan deret geometri murni tidak langsung berlaku.

2.3 Langkah 2: Derivasi Model Matematika — Rumus $\max(k)$

Dengan menuliskan nilai-nilai tersebut dalam bentuk yang lebih informatif:

k	$\max(k)$	Faktorisasi	Pola
1	7	$2 \cdot 4 - 1$	$(1 + 1) \cdot 2^{1+1} - 1$
2	23	$3 \cdot 8 - 1$	$(2 + 1) \cdot 2^{2+1} - 1$
3	63	$4 \cdot 16 - 1$	$(3 + 1) \cdot 2^{3+1} - 1$
4	159	$5 \cdot 32 - 1$	$(4 + 1) \cdot 2^{4+1} - 1$
5	383	$6 \cdot 64 - 1$	$(5 + 1) \cdot 2^{5+1} - 1$

Rumus Tertutup $\max(k)$:

$$\max(k) = (k + 1) \cdot 2^{k+1} - 1$$

Verifikasi:

- $k = 1$: $\max(1) = 2 \cdot 2^2 - 1 = 8 - 1 = 7 \checkmark$
- $k = 2$: $\max(2) = 3 \cdot 2^3 - 1 = 24 - 1 = 23 \checkmark$
- $k = 3$: $\max(3) = 4 \cdot 2^4 - 1 = 64 - 1 = 63 \checkmark$

- $k = 4$: $\max(4) = 5 \cdot 2^5 - 1 = 160 - 1 = 159 \checkmark$

2.4 Langkah 3: Analisis Batas dan Strategi Precompute

Batasan soal adalah $n \leq 10^9$. Karena $\max(k)$ tumbuh eksponensial, untuk $n \leq 10^9$ nilai k yang diperlukan tidak pernah melebihi 30. Ini memungkinkan strategi *precompute* yang sangat efisien.

Algoritma:

1. **Precompute:** Hitung dan simpan semua nilai $a[k] = \max(k) = (k + 1) \cdot 2^{k+1} - 1$ untuk $k = 1, 2, \dots, 30$ sebelum membaca input.
2. **Per Query:** Untuk setiap n , lakukan *linear scan* dari $k = 1$ ke atas, dan kembalikan k pertama yang memenuhi $n \leq a[k]$.

Analisis Kompleksitas STUDCHAN2:

- *Precompute*: $O(30) = O(1)$ — hanya 30 iterasi, dilakukan sekali
- Per query: $O(30) = O(1)$ — linear scan maksimal 30 langkah
- Total untuk T query: $O(T)$ — linear
- Space: $O(30) = O(1)$ — array tetap berukuran konstan

2.5 Langkah 4: Implementasi — Solusi Optimal C++

```

1 #include <cstdio>
2 using namespace std;
3 typedef long long ll;
4
5 int main() {
6     int t;
7     ll a[32], pow2 = 4ll, n;
8     scanf("%d", &t);
9
10    // Precompute: a[i-1] = i * 2^i - 1 = max(i-1)
11    // Inisialisasi: i=2, pow2=4=2^2 -> a[1] = 2*4-1 = 7 = max(1)
12    for (int i = 2; i < 31; i++) {
13        a[i-1] = (ll)i * pow2 - 1ll;
14        pow2 = pow2 * 2ll; // pow2 = 2^(i+1) untuk iterasi
15                               berikutnya
16    }
17
18    while (t--) {
19        scanf("%lld", &n);
20
21        // Linear scan: cari k terkecil dengan n <= max(k)
22        for (int i = 1; i <= 31; i++) {
23            if (n <= a[i]) {
24                printf("%d\n", i);
25                i = 99; // break: paksa loop berhenti
26            }
27        }
28    }
29 }

```

```

27     }
28
29     return 0;
30 }
```

Listing 1: SPOJ STUDCHAN2 — Implementasi C++

Penjelasan Komponen Kode:

1. **Precompute dengan pow2:** Inisialisasi $\text{pow2} = 4 = 2^2$ dan loop dimulai dari $i = 2$. Pada setiap iterasi: $a[i-1] = i \cdot \text{pow2} - 1 = i \cdot 2^i - 1$. Setelah itu pow2 digandakan: $\text{pow2} \leftarrow \text{pow2} \times 2 = 2^{i+1}$, siap untuk iterasi $i + 1$.
2. **Teknis $i = 99$ sebagai break:** Penghentian loop dilakukan dengan meng-assign $i = 99$, yang menyebabkan kondisi $i \leq 31$ langsung gagal. Dalam kode modern, penggunaan **break** lebih disarankan untuk keterbacaan.

2.6 Kaitan STUDCHAN2 dengan CBANK: Inversi Masalah

Aspek	CBANK (Pertemuan 2)	STUDCHAN2 (Pertemuan 3)
Arah	Diberikan N , cari $f(N)$	Diberikan n , cari k terkecil
Jenis	Forward (evaluasi rumus)	Inverse (pencarian batas)
Inti	Rumus polinomial derajat 3	$(k+1) \cdot 2^{k+1} - 1$
Strategi	Hitung langsung per query	Precompute lalu linear search
Kompleksitas	$O(1)$ per query	$O(1)$ per query

Pembahasan Kasus: SPOJ MONONUM — Monotone Numbers

3.1 Deskripsi Permasalahan

Diberikan bilangan bulat n , kita diminta menghitung *expected value* (nilai harapan) dari jumlah bilangan monotone (tidak menurun) yang panjangnya tepat n digit atau kurang.

Batasan Masukan:

- $T \leq 10^4$
- $n \leq 10^{15}$
- Keluaran berupa bilangan real dengan 6 angka di belakang koma.

Peringatan Penting

Tag soal ini adalah **Dynamic Programming**. Namun seperti yang akan kita buktikan, solusi optimal bukan DP, melainkan sebuah **rumus tertutup** $O(1)$ yang diturunkan dari observasi kombinatorik.

3.2 Langkah 1: Memahami Inti Rumus dari Kode

Kode solusi yang diberikan:

```

1 #include <cstdio>
2 using namespace std;
```

```

3 typedef long long ll;
4
5 int main() {
6     int t;
7     scanf("%d", &t);
8
9     while (t--) {
10         ll n;
11         scanf("%lld", &n);
12
13         double val = 1.0;
14
15         // val = (n+1)(n+2)(n+3)(n+4)(n+5)(n+6)(n+7)(n+8)
16         for (int i = 1; i <= 8; i++) {
17             val *= (double)(n + i);
18         }
19
20         // result = (n+9)/9 - 40320/val
21         double result = ((double)n + 9.0) / 9.0 - (40320.0 / val);
22
23         printf("%.6lf\n", result);
24     }
25
26     return 0;
27 }

```

Listing 2: SPOJ MONONUM — Implementasi C++

3.3 Langkah 2: Dekomposisi Matematis Rumus

Dari kode, terdapat dua komponen utama:

1. Variabel **val** (rising factorial):

$$\text{val} = \prod_{i=1}^8 (n+i) = (n+1)(n+2)(n+3)(n+4)(n+5)(n+6)(n+7)(n+8)$$

2. Konstanta **40320**: $40320 = 8! = 1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7 \times 8$

3. Rumus hasil:

$$\text{result} = \frac{n+9}{9} - \frac{8!}{\prod_{i=1}^8 (n+i)}$$

3.4 Langkah 3: Penyederhanaan via Koefisien Binomial

Menyederhanakan suku kedua:

$$\frac{8!}{\prod_{i=1}^8 (n+i)} = \frac{8! \cdot n!}{(n+8)!} = \frac{1}{\binom{n+8}{8}}$$

Menyederhanakan suku pertama dengan identitas koefisien binomial:

$$\frac{n+9}{9} = \frac{\binom{n+9}{9}}{\binom{n+8}{8}}$$

Substitusi ke rumus menghasilkan bentuk akhir yang elegan:

Rumus MONONUM dalam Koefisien Binomial:

$$f(n) = \frac{\binom{n+9}{9} - 1}{\binom{n+8}{8}}$$

3.5 Langkah 4: Interpretasi Kombinatorik

Definisi

Stars and Bars: Jumlah cara mendistribusikan n objek identik ke dalam k kotak berbeda (boleh kosong) adalah $\binom{n+k-1}{k-1}$. Ekuivalen dengan menghitung multiset berukuran n dari k elemen.

Dalam konteks bilangan monotone (tidak menurun), $\binom{n+k}{k}$ menghitung jumlah barisan panjang n dari alfabet berukuran $k+1$ yang tidak menurun. Jumlah bilangan d -digit (setiap digit $\in \{0, 1, \dots, 9\}$) yang digit-digitnya membentuk barisan tidak menurun adalah:

$$\binom{d+9}{9}$$

Ini merupakan jumlah cara memilih multiset berukuran d dari 10 simbol $\{0, 1, \dots, 9\}$.

3.6 Langkah 5: Verifikasi Numerik

Contoh 1 ($n = 1$):

$$f(1) = \frac{\binom{10}{9} - 1}{\binom{9}{8}} = \frac{10 - 1}{9} = \frac{9}{9} = 1.000000\checkmark$$

Contoh 2 ($n = 2$):

$$f(2) = \frac{\binom{11}{9} - 1}{\binom{10}{8}} = \frac{55 - 1}{45} = \frac{54}{45} = 1.200000\checkmark$$

Contoh 3 ($n = 3$):

$$f(3) = \frac{\binom{12}{9} - 1}{\binom{11}{8}} = \frac{220 - 1}{165} = \frac{219}{165} = \frac{73}{55} \approx 1.327273\checkmark$$

3.7 Langkah 6: Kenapa Bukan Dynamic Programming?

Pendekatan DP yang “wajar” mendefinisikan:

$dp[i][j]$ = jumlah bilangan i -digit yang digit terakhirnya adalah j

dengan transisi $dp[i][j] = \sum_{d=0}^j dp[i-1][d]$.

Kompleksitas: $O(n \times 10 \times 10) = O(100n)$ per query. Untuk $n \leq 10^{15}$, ini sama sekali tidak *feasible*.

Aspek	DP	Rumus Tertutup
Time per query	$O(100n)$	$O(1)$
Feasible untuk $n = 10^{15}$	Tidak	Ya
Memori	$O(n \times 10)$	$O(1)$
Kemungkinan overflow	Tinggi	Rendah (floating-point)

3.8 Langkah 7: Analisis Kompleksitas MONONUM

Komponen	Kompleksitas	Keterangan
Hitung <code>val</code> (loop 8x)	$O(8) = O(1)$	Jumlah iterasi tetap
Hitung <code>result</code>	$O(1)$	Aritmetika floating-point
Per query total	$O(1)$	
Total untuk T query	$O(T)$	Linear
Space complexity	$O(1)$	Hanya variabel skalar

Sangat efisien: bahkan untuk $T = 10^4$ dan $n = 10^{15}$, program berjalan hampir instan.

Kesimpulan

Dari keseluruhan materi yang telah dipelajari, beberapa butir penting dapat dirangkum sebagai berikut:

1. **STUDCHAN2 sebagai Masalah Invers:** Kebalikan dari evaluasi fungsi adalah pencarian batas. Kunci solusinya adalah mengenali bahwa fungsi $\max(k) = (k + 1) \cdot 2^{k+1} - 1$ tumbuh eksponensial, sehingga k yang diperlukan sangat kecil (≤ 30 untuk $n \leq 10^9$). Precompute + linear scan memberikan solusi $O(T)$ yang sangat efisien.
2. **MONONUM sebagai Studi Kasus Anti-DP:** Rumus $f(n) = \frac{\binom{n+9}{9} - 1}{\binom{n+8}{8}}$ yang diturunkan dari prinsip Stars and Bars menghasilkan solusi $O(1)$ per query — jauh lebih baik dari DP $O(100n)$ yang tidak *feasible* untuk $n \leq 10^{15}$.
3. **Kekuatan Observasi:** Observasi sederhana pada tabel nilai kecil, dikombinasikan dengan pemahaman kombinatorik dasar (Stars and Bars, koefisien binomial, *rising factorial*), mampu menghasilkan rumus tertutup yang elegan untuk masalah yang tampaknya kompleks.
4. **Worst-Case Thinking:** Selalu rancang solusi untuk input terbesar. Untuk MONONUM, $n = 10^{15}$ langsung mengeliminasi semua pendekatan iteratif dan memaksa kita menemukan solusi $O(1)$.

5. **Metodologi Pemecahan Masalah yang Sistematis:** Ketiga studi kasus mendemonstrasikan alur berpikir terstruktur yang dapat diterapkan secara universal:
Pahami Soal $\rightarrow$ Abstraksi $\rightarrow$ Observasi Pola $\rightarrow$ Pemodelan Matematis $\rightarrow$ Implementasi
6. **Prinsip “Jangan Percaya Tag Soal”:** Tag soal adalah petunjuk awal, bukan solusi. Tag “Dynamic Programming” pada MONONUM menyesatkan: solusi optimal adalah rumus tertutup $O(1)$, bukan DP $O(n)$. Ini menunjukkan bahwa observasi mendalam dan pemahaman kombinatorik sering kali jauh lebih kuat daripada mengikuti kategori secara membabi buta.

Pustaka

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C. (2022). *Introduction to Algorithms, 4th edition*. MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- [2] Rosen, K.H. (2019). *Discrete Mathematics and Its Applications, 8th edition*. McGraw-Hill Education. <https://www.mheducation.com/highered/product/discrete-mathematics-applications-rosen/M9781259676512.html>
- [3] Wirth, N. (1976). *Algorithms + Data Structures = Programs*. Prentice-Hall. <https://dl.acm.org/doi/book/10.5555/540029>
- [4] SPOJ. (n.d.). STUDCHAN2 — Student Chains 2. <https://www.spoj.com/problems/STUDCHAN2/>
- [5] SPOJ. (n.d.). MONONUM — Monotone Numbers. <https://www.spoj.com/problems/MONONUM/>
- [6] SPOJ. (n.d.). CBANK — Charu and Coin Distribution. <https://www.spoj.com/problems/CBANK/>

Bukti Submission

GilbranNRP134: submissions
Student Chains

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35594155	2026-03-11 15:20:10	Student Chains	accepted sbl - Monotone R	0.01	5.2M	CPP14

Gambar 1: Accepted: SPOJ STUDCHAN2 — Student Chains 2

GilbranNRP134: submissions
Monotonous numbers

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35594258	2026-03-11 15:48:24	Monotonous numbers	accepted sbl - Monotone R	0.01	5.2M	CPP14

Gambar 2: Accepted: SPOJ MONONUM — Monotone Numbers